# 1 Syntax von Programmiersprachen

## Syntax ("Lehre vom Satzbau")

- formale Beschreibung des Aufbaus der "Worte" und "Sätze", die zu einer Sprache gehören;
- im Falle einer Programmier-Sprache Festlegung, wie Programme aussehen müssen.

# Hilfsmittel bei natürlicher Sprache

- Wörterbücher;
- Rechtschreibregeln, Trennungsregeln, Grammatikregeln;
- Ausnahme-Listen;
- Sprach-“Gefühl”.

# Hilfsmittel bei Programmiersprachen

- Listen von Schlüsselworten wie `if`, `int`, `else`, `while` ...
- Regeln, wie einzelne Worte (Tokens) z.B. Namen gebildet werden.

  Frage:

  Ist `x10` ein zulässiger Name für eine Variable?

  oder `_ab$` oder `A#B` oder `0A?B` ...
- Grammatikregeln, die angeben, wie größere Komponenten aus kleineren aufgebaut werden.

  Frage:

  Ist ein `while`-Statement im `else`-Teil erlaubt?

- Kontextbedingungen.

  Beispiel:

  Eine Variable muss erst deklariert sein, bevor sie verwendet wird.

$\Longrightarrow$ formalisierter als natürliche Sprache

$\Longrightarrow$ besser für maschinelle Verarbeitung geeignet

# Semantik ("Lehre von der Bedeutung")

- Ein Satz einer (natürlichen) Sprache verfügt zusätzlich über eine Bedeutung, d.h. teilt einem Hörer/Leser einen Sachverhalt mit (↑Information)

- Ein Satz einer Programmiersprache, d.h. ein Programm verfügt ebenfalls über eine Bedeutung ...

Die Bedeutung eines Programms ist

- alle möglichen Ausführungen der beschriebenen Berechnung (↑operationelle Semantik); oder
- die definierte Abbildung der Eingaben auf die Ausgaben (↑denotationelle Semantik).

Die Bedeutung eines Programms ist

- alle möglichen Ausführungen der beschriebenen Berechnung (↑operationelle Semantik); oder
- die definierte Abbildung der Eingaben auf die Ausgaben (↑denotationelle Semantik).

## Achtung!

Ist ein Programm syntaktisch korrekt, heißt das noch lange nicht, dass es auch das "richtige" tut, d.h. semantisch korrekt ist !!!

## 1.1 Reservierte Wörter

- `int`

  → Bezeichner für Basis-Typen;

- `if`, `else`, `while`

  → Schlüsselwörter aus Programm-Konstrukten;

- `(`,`)`, `"`,`'`, `{`,`}`, `,`,`;`, `_`

  → Sonderzeichen.

## 1.2 Was ist ein erlaubter Name?

Schritt 1: Angabe der erlaubten Zeichen:

$$
\begin{array}{lcl}
\text{letter} & ::= & \texttt{\$} \mid \texttt{_} \mid \texttt{a} \mid \ldots \mid \texttt{z} \mid \texttt{A} \mid \ldots \mid \texttt{Z} \\
\text{digit} & ::= & 0 \mid \ldots \mid 9
\end{array}
$$

## 1.2 Was ist ein erlaubter Name?

**Schritt 1:** Angabe der erlaubten Zeichen:

$$
\begin{array}{lcl}
\text{letter} & ::= & \texttt{\$} \mid \texttt{_} \mid \texttt{a} \mid \ldots \mid \texttt{z} \mid \texttt{A} \mid \ldots \mid \texttt{Z} \\
\text{digit} & ::= & \texttt{0} \mid \ldots \mid \texttt{9}
\end{array}
$$

- letter und digit bezeichnen Zeichenklassen, d.h. Mengen von Zeichen, die gleich behandelt werden.
- Das Symbol "|" trennt zulässige Alternativen.
- Das Symbol "..." repräsentiert die Faulheit, alle Alternativen wirklich aufzuzählen.

## Schritt 2: Angabe der Anordnung der Zeichen:

name ::= letter ( letter | digit )*

- Erst kommt ein Zeichen der Klasse letter, dann eine (eventuell auch leere) Folge von Zeichen entweder aus letter oder aus digit.
- Der Operator "*" bedeutet "beliebig ofte Wiederholung" ("weglassen" ist 0-malige Wiederholung).
- Der Operator "*" ist ein Postfix-Operator. Das heißt, er steht hinter seinem Argument.

# Beispiele

- `$178`
  `Das$ist$kein$Name`
  `x`
  `$`
  `Password`

... sind legale Namen.

- `5ABC`

  `-`

  `!Hallo!`

  `x'`

  `-178`

... sind keine legalen Namen.

- `5ABC`

  `-`

  `!Hallo!`

  `x'`

  `-178`

... sind keine legalen Namen.

## Achtung

Reservierte Wörter sind als Namen verboten !!!

## 1.3 Ganze Zahlen

Werte, die direkt im Programm stehen, heißen Konstanten.

Ganze nichtnegative dezimale Zahlen bestehen aus einer nichtleeren Folge von Ziffern – ohne führende Nullen:

$$\text{number} \quad ::= \quad 0 \quad | \quad (1 | \ldots | 9)\ \text{digit}^*$$

# Beispiele

- `17`
  `12490`
  `42`
  `0`
  `70`

  ... sind alles legale `int`-Konstanten.

- `08`
  `"Hello World!"`
  `0.5e+128`

  ... sind keine `int`-Konstanten.

Ausdrücke, die aus Zeichen (-klassen) mithilfe von

| (Alternative)

* (Iteration)

(Konkatenation) sowie

? (Option)

... aufgebaut sind, heißen reguläre Ausdrücke[a] (↑Automatentheorie).

Der Postfix-Operator "?" besagt, dass das Argument eventuell auch fehlen darf, d.h. einmal oder keinmal vorkommt.

[a] Gelegentlich sind auch $\epsilon$, d.h. das "leere Wort" sowie $\emptyset$, d.h. die leere Menge zugelassen.

Reguläre Ausdrücke reichen zur Beschreibung einfacher Mengen von Worten aus.

- ( letter letter )*
  – alle Wörter gerader Länge (über `$,a,...,z,A,...,Z`);
- letter* `test` letter*
  – alle Wörter, die das Teilwort `test` enthalten;
- `$` digit* `17`
  – alle Wörter, die mit `$` anfangen, dann eine beliebige Folge von Ziffern aufweisen, die mit `17` aufhört;
- $$\begin{array}{lcl} \text{exp} & ::= & (\texttt{e}|\texttt{E})(\texttt{+}|\texttt{-})?\ \text{digit digit}^* \\ \text{double} & ::= & \text{digit digit}^*\ \text{exp} \quad | \\ & & \text{digit}^*\ (\text{digit}\ \texttt{.}\ |\ \texttt{.}\ \text{digit})\ \text{digit}^*\ \text{exp}? \end{array}$$
  – alle Gleitkomma-Zahlen ...

Identifizierung von

- reservierten Wörtern,
- Namen,
- Konstanten

Ignorierung von

- White Space,
- Kommentaren

... erfolgt in einer ersten Phase (↑Scanner)

$\Longrightarrow$ Input wird mit regulären Ausdrücken verglichen und dabei in Wörter ("Tokens") aufgeteilt.

In einer zweiten Phase wird die Struktur des Programms analysiert (↑Parser).

## 1.4 Struktur von Programmen

Programme sind hierarchisch aus Komponenten aufgebaut. Für jede Komponente geben wir Regeln an, wie sie aus anderen Komponenten zusammengesetzt sein können.

| | | |
|---|---|---|
| program | ::= | decl* stmt* |
| decl | ::= | type name ( , name )* ; |
| type | ::= | `int` |

## 1.4 Struktur von Programmen

Programme sind hierarchisch aus Komponenten aufgebaut. Für jede Komponente geben wir Regeln an, wie sie aus anderen Komponenten zusammengesetzt sein können.

| | | |
|---|---|---|
| program | ::= | decl* stmt* |
| decl | ::= | type name ( , name )* ; |
| type | ::= | `int` |

- Ein Programm besteht aus einer Folge von Deklarationen, gefolgt von einer Folge von Statements.
- Eine Deklaration gibt den Typ an, hier: `int`, gefolgt von einer Komma-separierten Liste von Variablen-Namen.

```
stmt    ::=    ; | { stmt* } |
               name = expr; | name = readInt(); | write( expr ); |
               if ( cond ) stmt |
               if ( cond ) stmt else stmt |
               while ( cond ) stmt
```

- Ein Statement ist entweder "leer" (d.h. gleich `;`) oder eine geklammerte Folge von Statements;
- oder eine Zuweisung, eine Lese- oder Schreib-Operation;
- eine (einseitige oder zweiseitige) bedingte Verzweigung;
- oder eine Schleife.

$$
\begin{array}{lcl}
\text{expr} & ::= & \text{number} \mid \text{name} \mid \texttt{(}\ \text{expr}\ \texttt{)} \mid \\
 & & \text{unop expr} \mid \text{expr binop expr} \\
\text{unop} & ::= & \texttt{-} \\
\text{binop} & ::= & \texttt{-} \mid \texttt{+} \mid \texttt{*} \mid \texttt{/} \mid \texttt{\%}
\end{array}
$$

- Ein Ausdruck ist eine Konstante, eine Variable oder ein geklammerter Ausdruck
- oder ein unärer Operator, angewandt auf einen Ausdruck,
- oder ein binärer Operator, angewand auf zwei Argument-Ausdrücke.
- Einziger unärer Operator ist (bisher) die Negation.
- Mögliche binäre Operatoren sind Addition, Subtraktion, Multiplikation, (ganz-zahlige) Division und Modulo.

| | | |
|---|---|---|
| cond | ::= | `true` \| `false` \| `(` cond `)` \| |
| | | expr comp expr \| |
| | | bunop cond \| cond bbinop cond |
| comp | ::= | `==` \| `!=` \| `<=` \| `<` \| `>=` \| `>` |
| bunop | ::= | `!` |
| bbinop | ::= | `&&` \| `\|\|` |

- Bedingungen unterscheiden sich dadurch von Ausdrücken, dass ihr Wert nicht vom Typ `int` ist sondern `true` oder `false` (ein Wahrheitswert – vom Typ `boolean`).
- Bedingungen sind darum Konstanten, Vergleiche
- oder logische Verknüpfungen anderer Bedingungen.

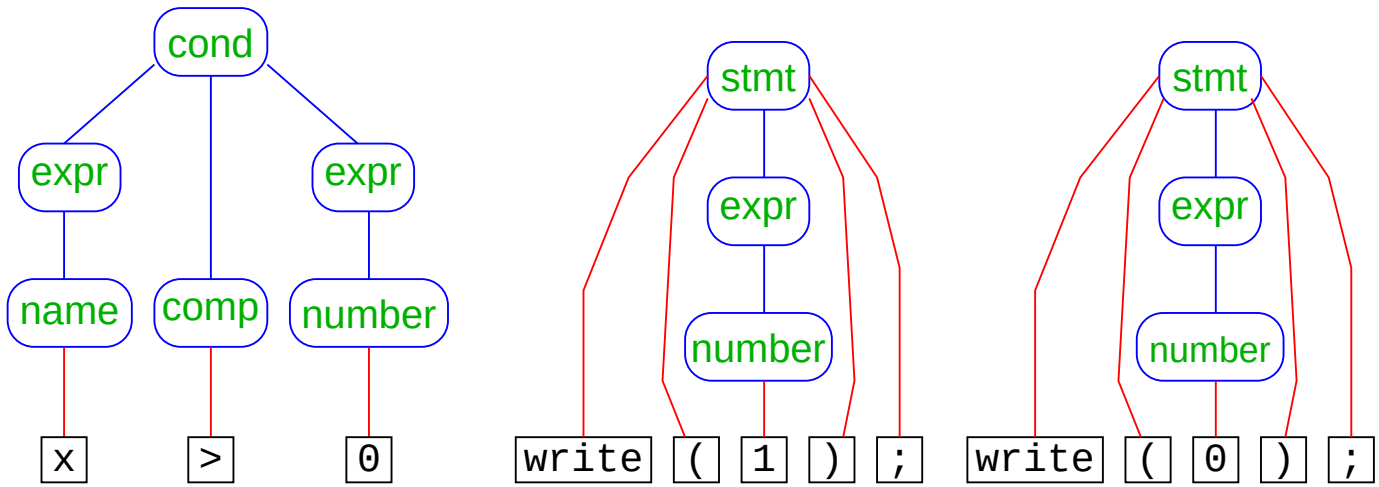
Puh!!! Geschafft ...

## Beispiel

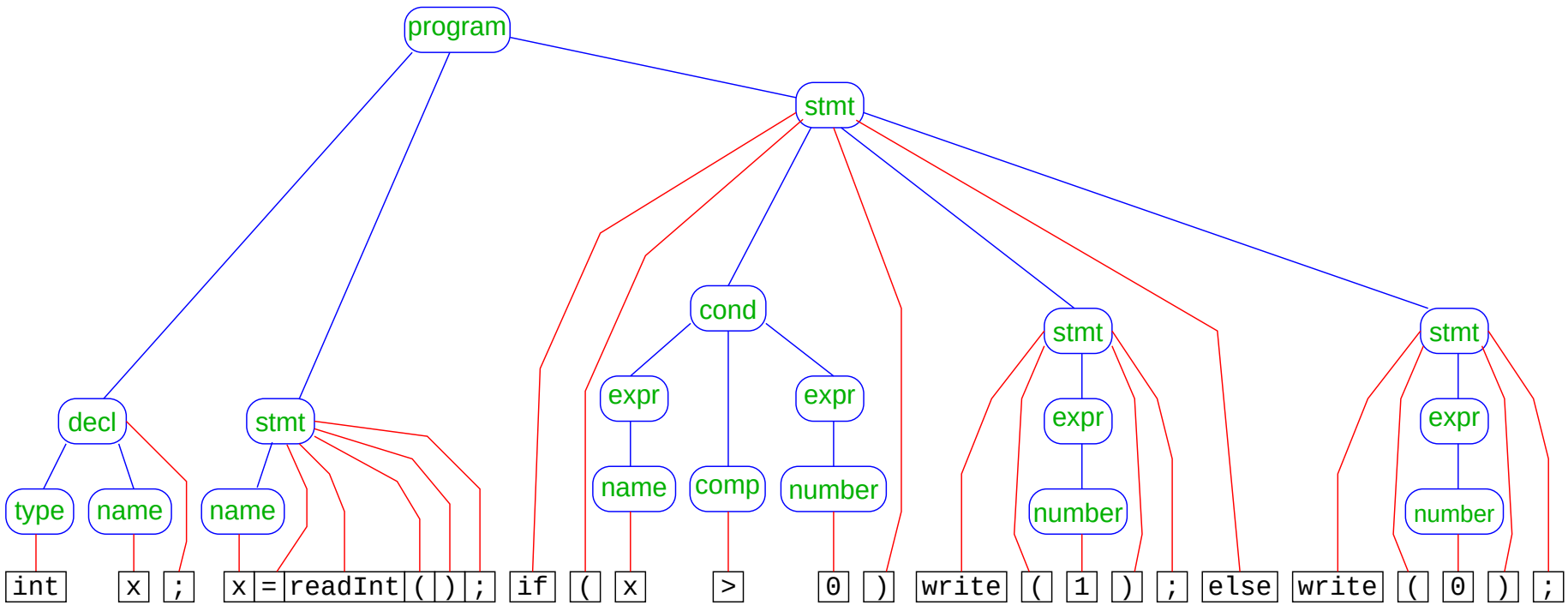
```
int x;
x = readInt();
if (x > 0)
    write(1);
else
    write(0);
```

Die hierarchische Untergliederung von Programm-Bestandteilen veranschaulichen wir durch Syntax-Bäume:

Syntax-Bäume für `x > 0` sowie `write(0);` und `write(1);`



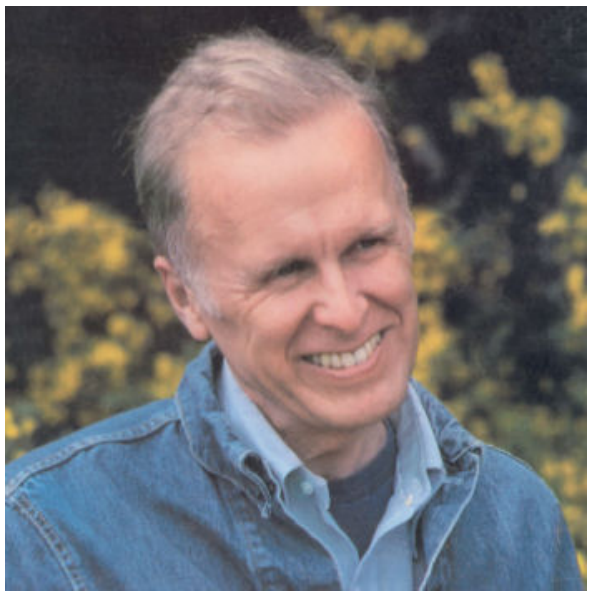

| | |
|---|---|
| Blätter: | Wörter/Tokens |
| innere Knoten: | Namen von Programm-Bestandteilen |


program
stmt
decl
stmt
cond
stmt
stmt
type
name
name
expr
name
comp
expr
number
expr
number
expr
number
int
x
;
x
=
readInt
(
)
;
if
(
x
>
0
)
write
(
1
)
;
else
write
(
0
)
;

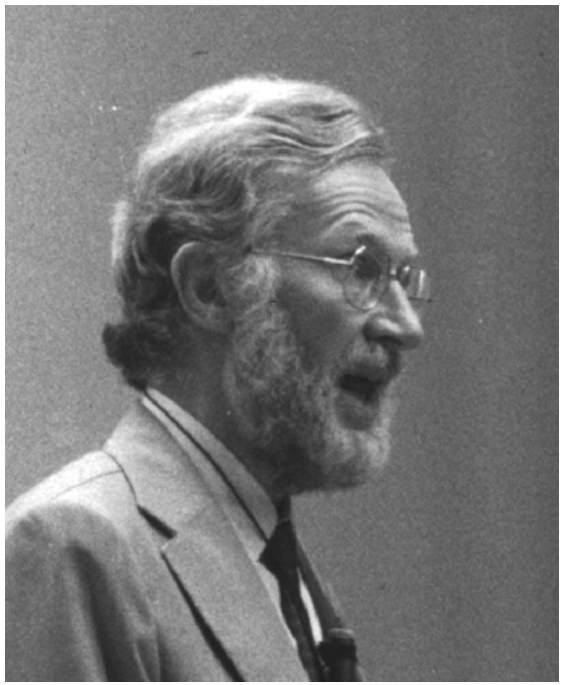
# Bemerkungen

- Die vorgestellte Methode der Beschreibung von Syntax heißt EBNF-Notation (Extended Backus Naur Form Notation).
- Ein anderer Name dafür ist erweiterte kontextfreie Grammatik (↑Linguistik, Automatentheorie).
- Linke Seiten von Regeln heißen auch Nicht-Terminale.
- Tokens heißen auch Terminale.

Noam Chomsky,
MIT



John Backus, IBM
Turing Award
(Erfinder von Fortran)



Peter Naur,
Turing Award
(Erfinder von Algol60)

## Achtung

- Die regulären Ausdrücke auf den rechten Regelseiten können sowohl Terminale wie Nicht-Terminale ohne Einschränkung enthalten.
- Deshalb sind kontextfreie Grammatiken mächtiger als reguläre Ausdrücke.

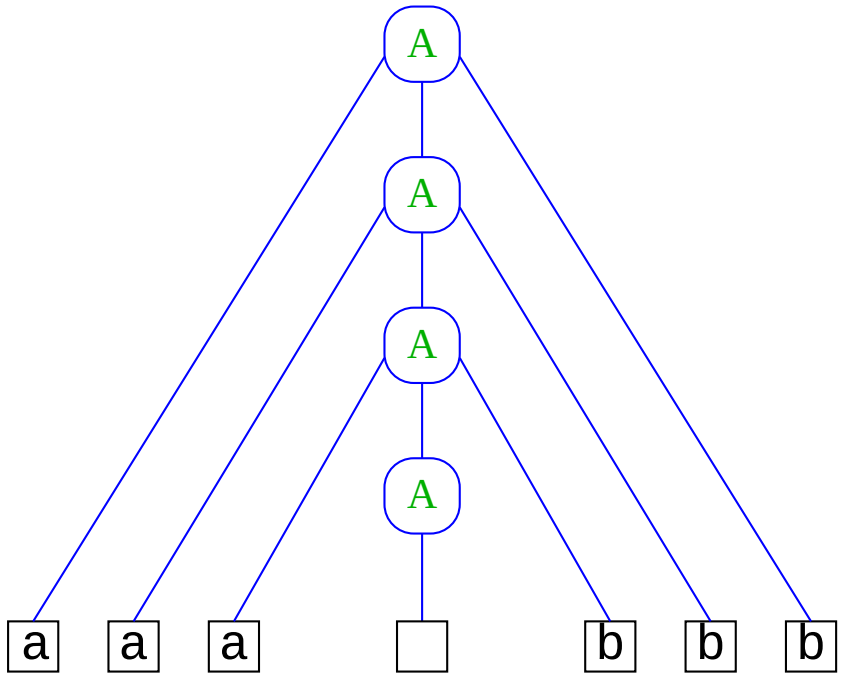
## Beispiel

$$\mathcal{L} = \{\epsilon, \mathtt{ab}, \mathtt{aabb}, \mathtt{aaabbb}, \ldots\}$$

lässt sich mithilfe einer Grammatik beschreiben:

$$\mathsf{A} \quad ::= \quad (\ \mathtt{a}\ \mathsf{A}\ \mathtt{b}\ )?$$

Syntax-Baum für das Wort `aaabbb` :



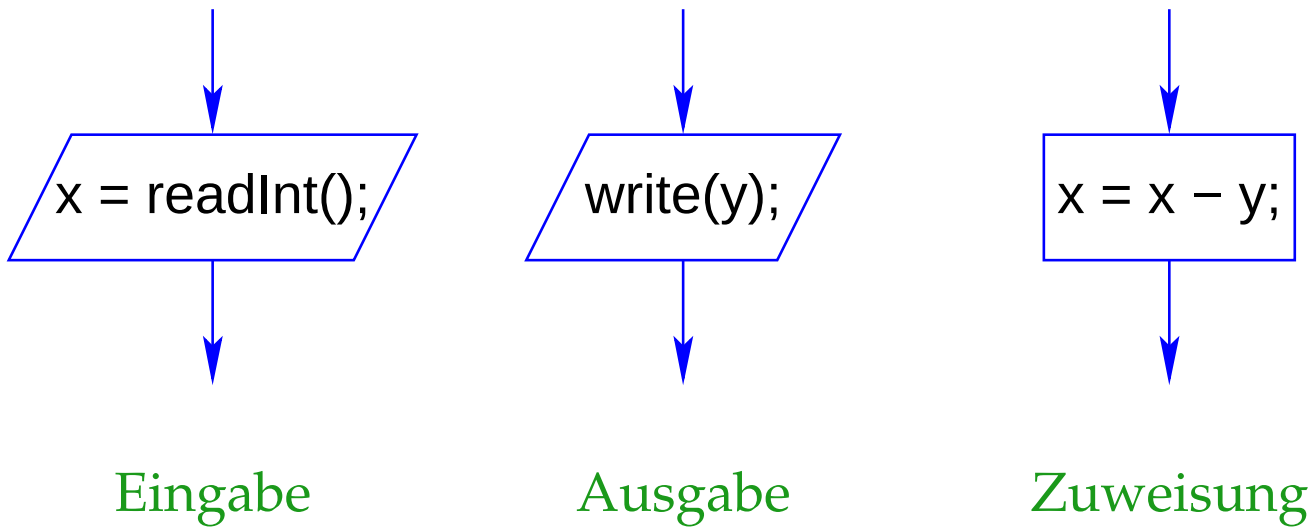

Für $\mathcal{L}$ gibt es aber keinen regulären Ausdruck!!! (↑Automatentheorie)

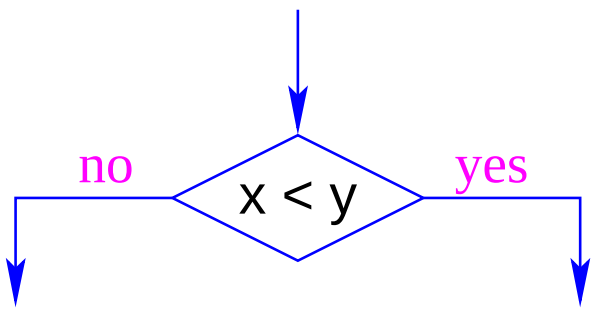
# 2 Kontrollfluss-Diagramme

In welcher Weise die Operationen eines Programms nacheinander ausgeführt werden, lässt sich anschaulich mithilfe von Kontrollfluss-Diagrammen darstellen.

## Ingredienzien

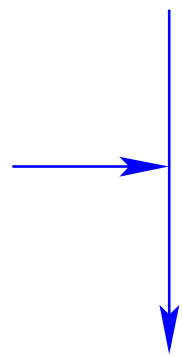
Startknoten Endknoten


x = readInt();
write(y);
x = x − y;
Eingabe
Ausgabe
Zuweisung




bedingte Verzweigung

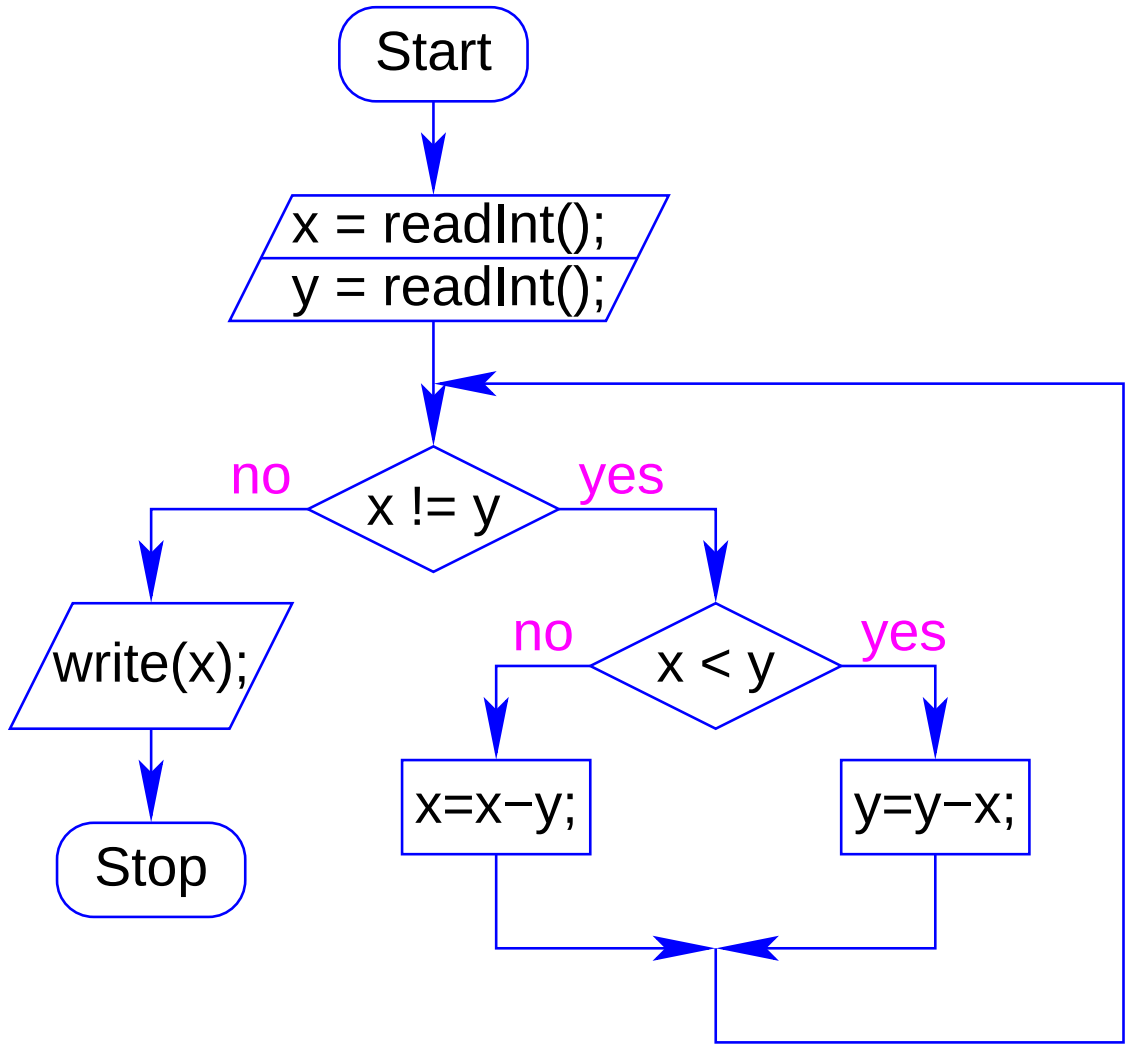
Kante



Zusammenlauf

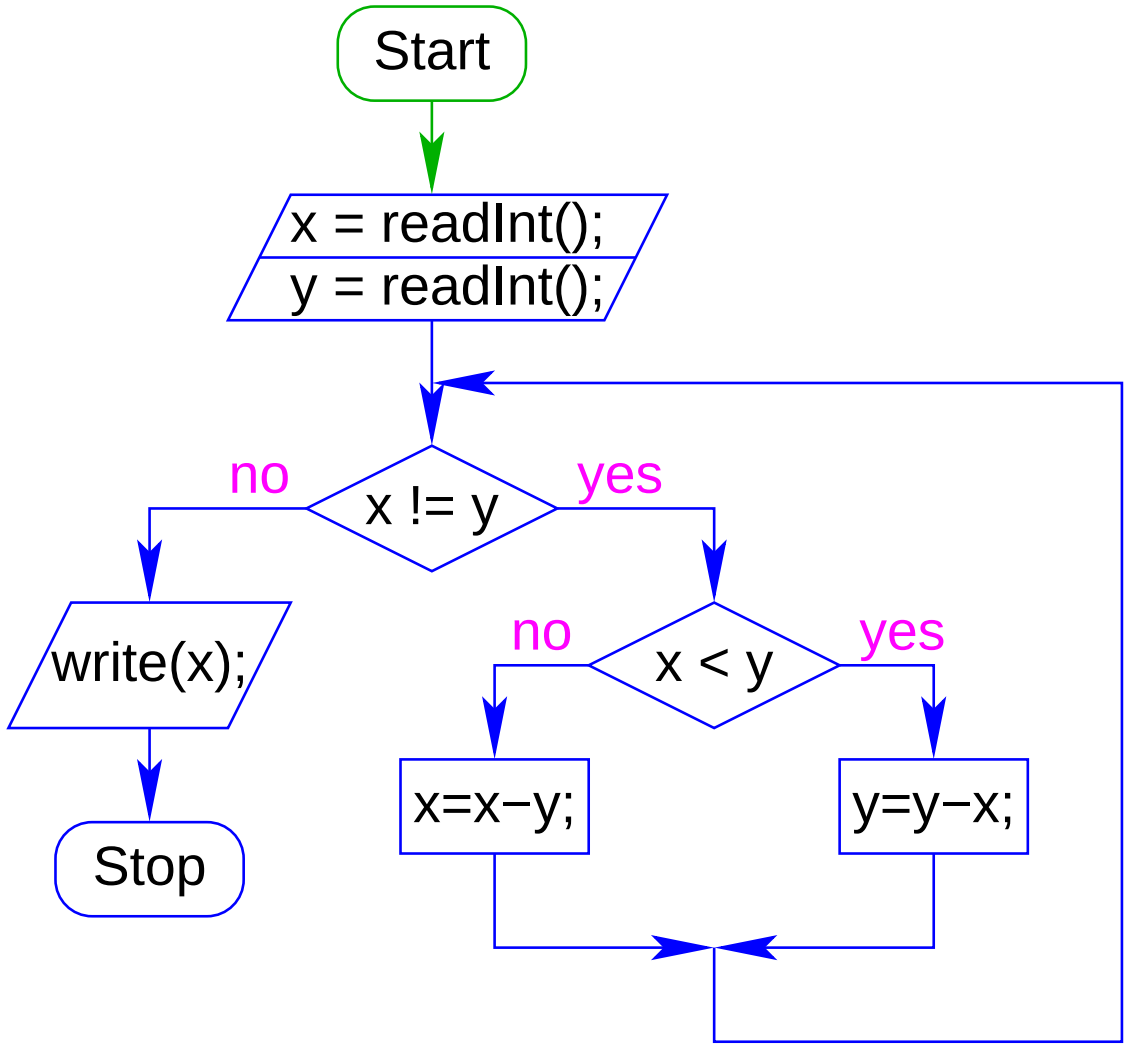
# Beispiel

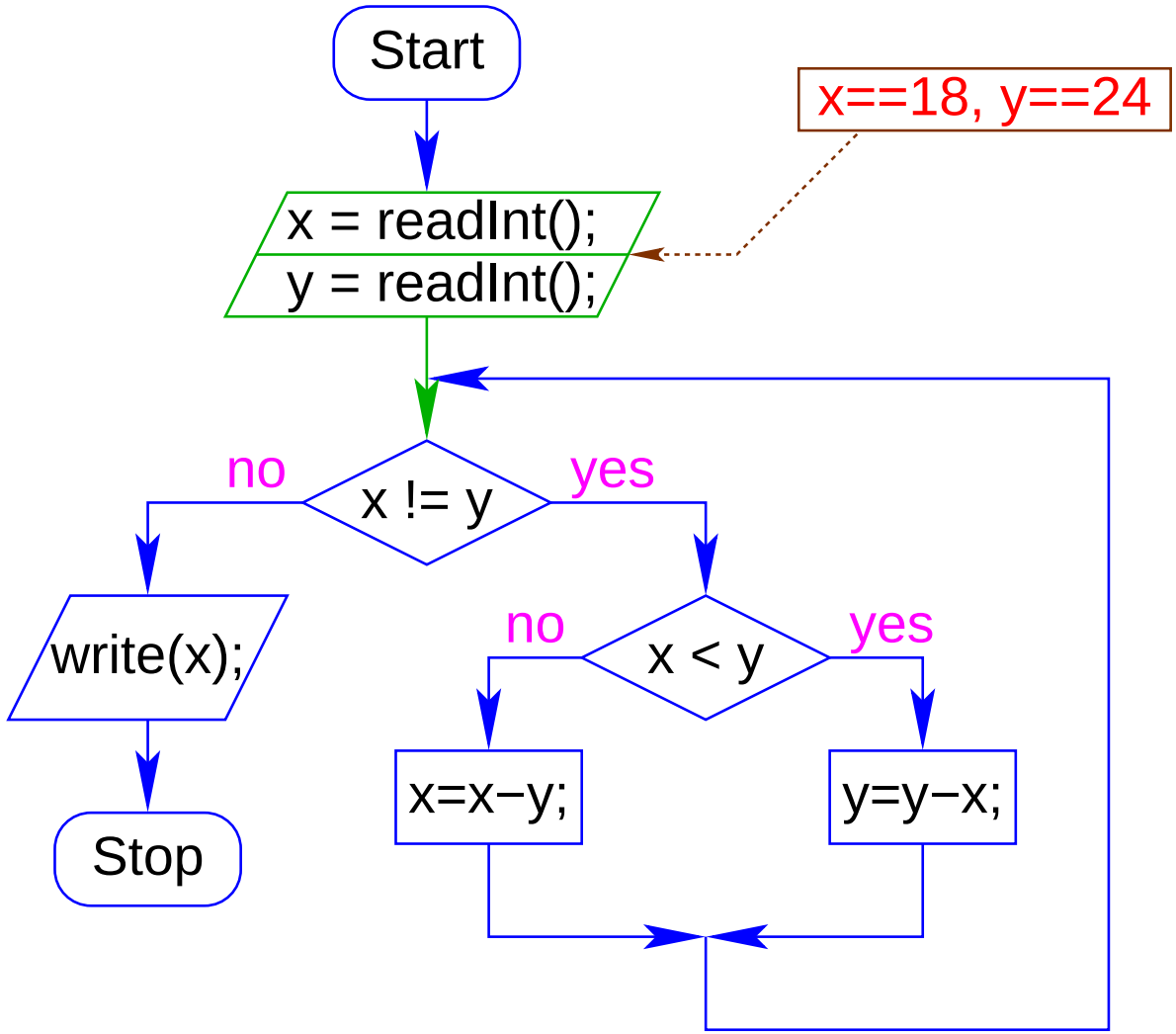
```
int x, y;
x = readInt();
y = readInt();
while (x != y)
    if (x < y)
        y = y - x;
    else
        x = x - y;
write(x);
```

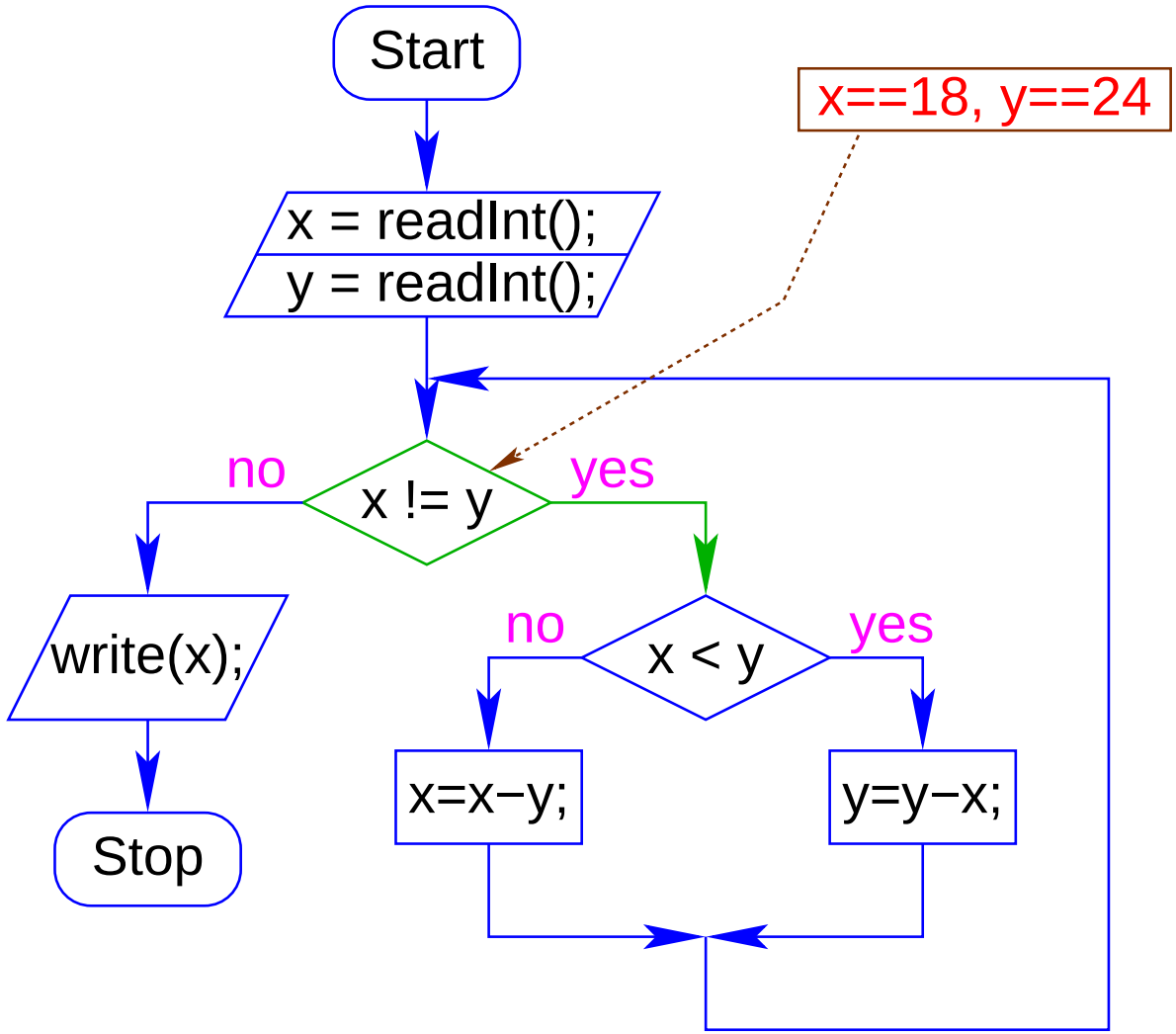


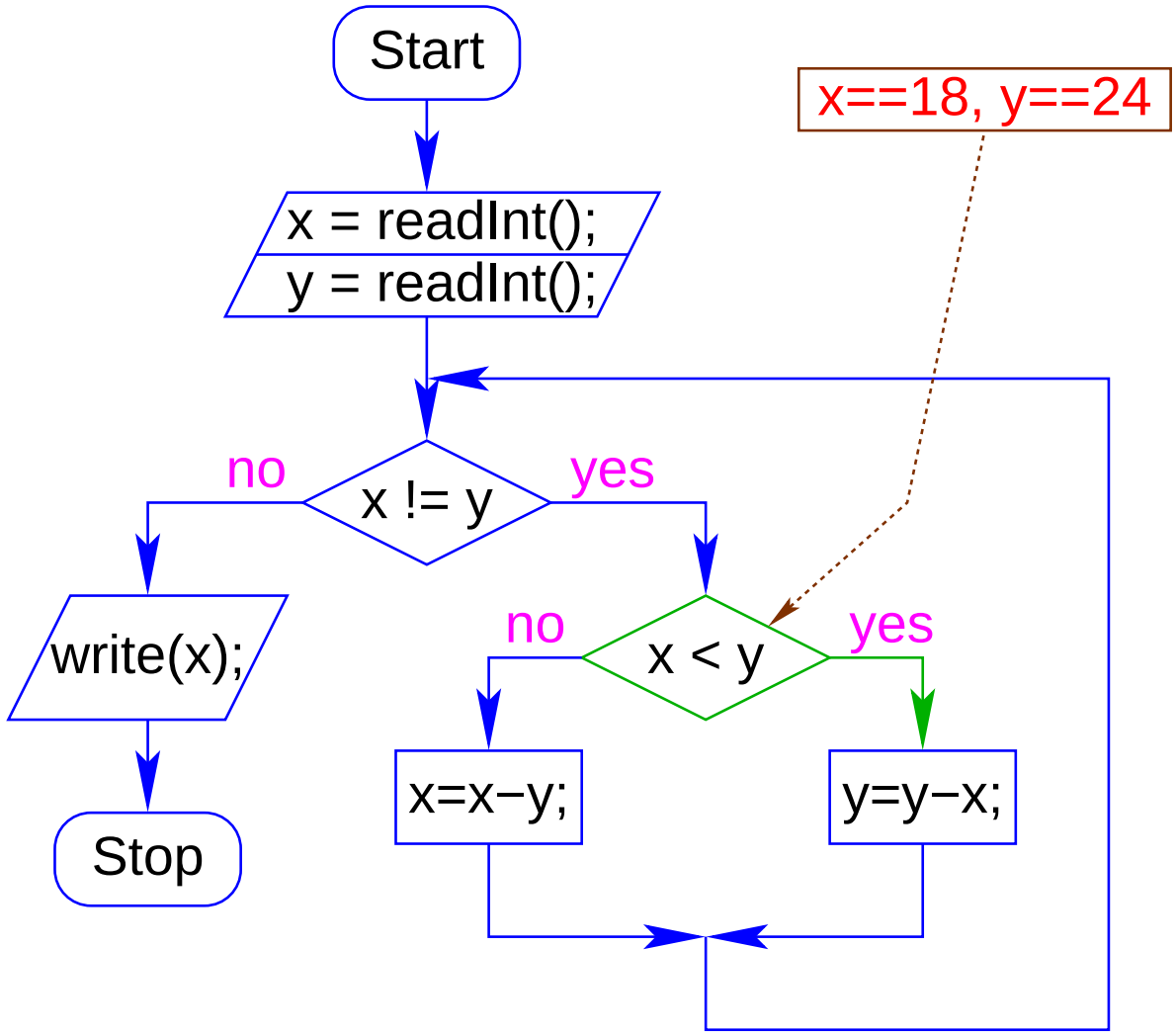
- Die Ausführung des Programms entspricht einem Pfad durch das Kontrollfluss-Diagramm vom Startknoten zum Endknoten.
- Die Deklarationen von Variablen muss man sich am Startknoten vorstellen.
- Die auf dem Pfad liegenden Knoten (außer dem Start- und Endknoten) sind die dabei auszuführenden Operationen bzw. auszuwertenden Bedingungen.
- Um den Nachfolger an einem Verzweigungsknoten zu bestimmen, muss die Bedingung für die aktuellen Werte der Variablen ausgewertet werden.

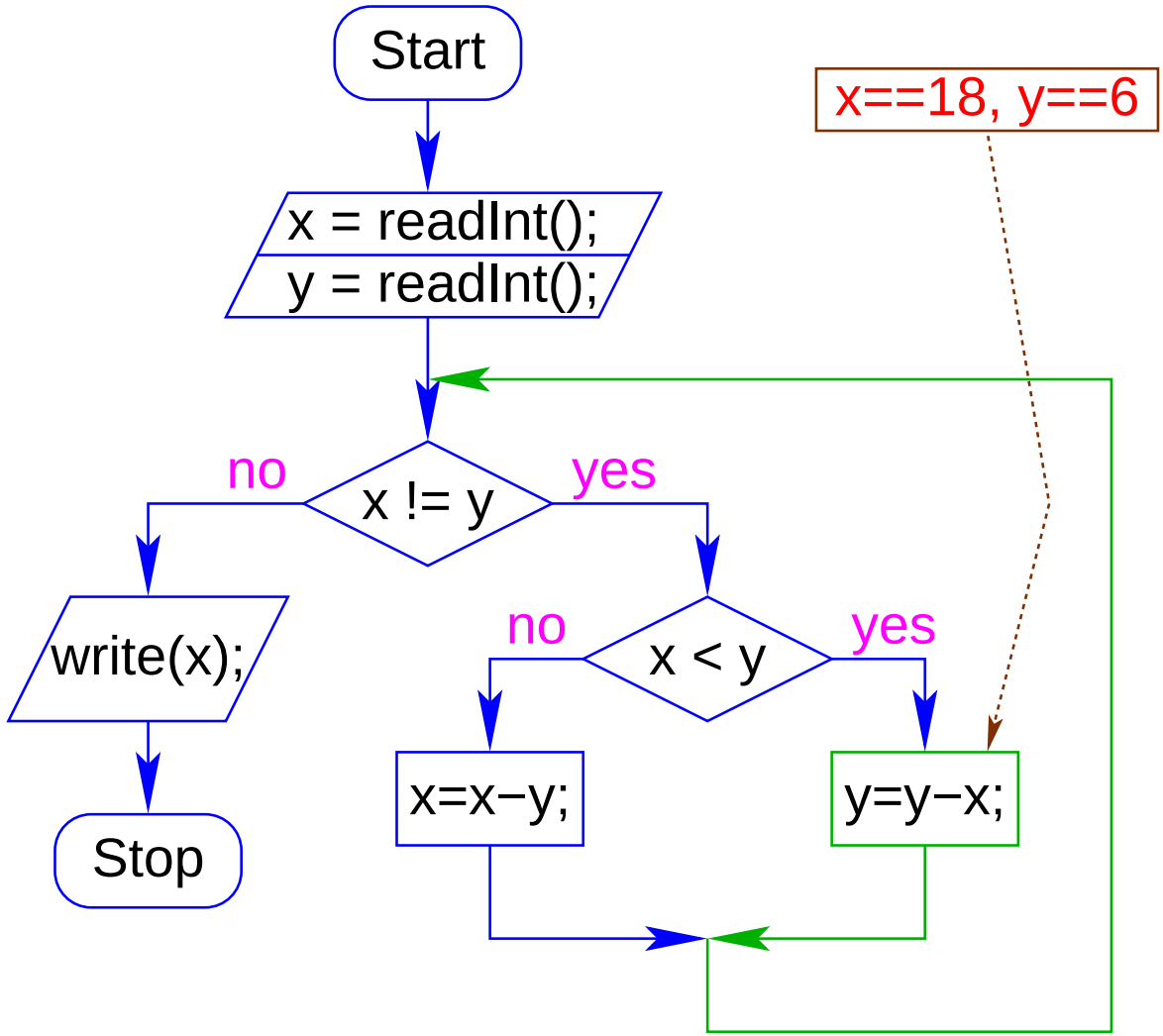
$\Longrightarrow$ operationelle Semantik

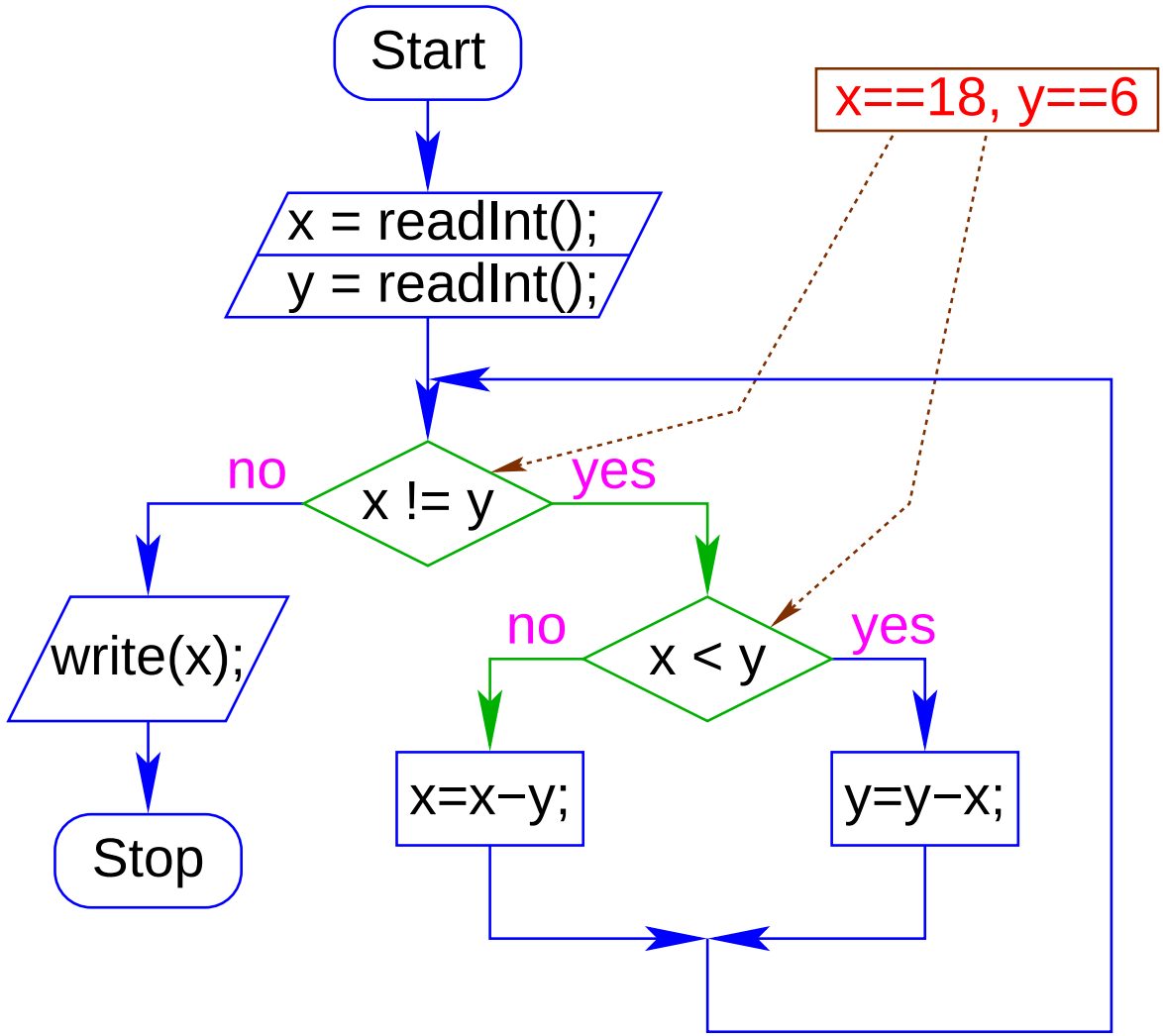

Start
x = readInt();
y = readInt();
x != y
no
yes
write(x);
Stop
x < y
no
yes
x=x−y;
y=y−x;

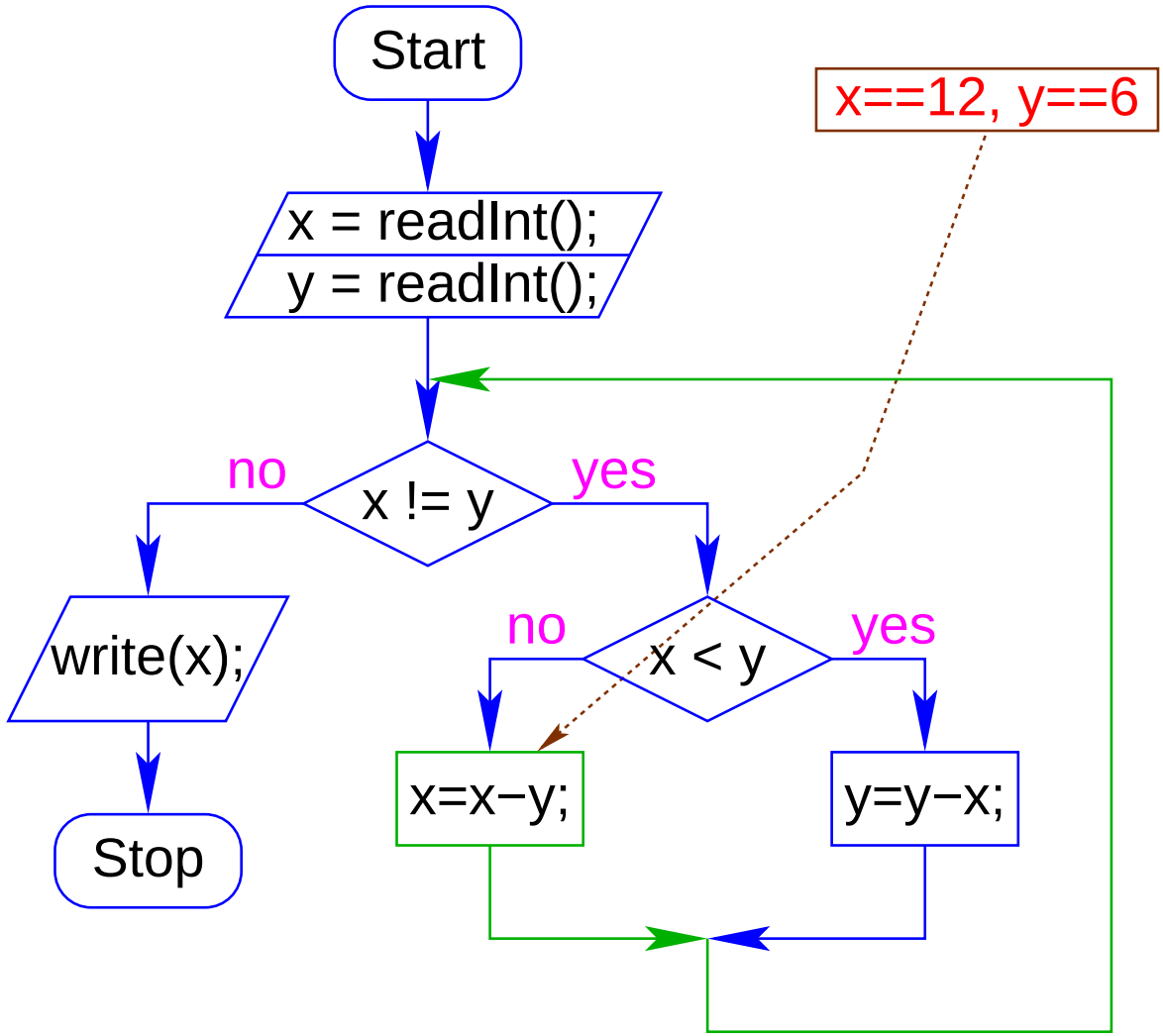

Start
x==18, y==24
x = readInt();
y = readInt();
no
x != y
yes
write(x);
Stop
no
x < y
yes
x=x−y;
y=y−x;

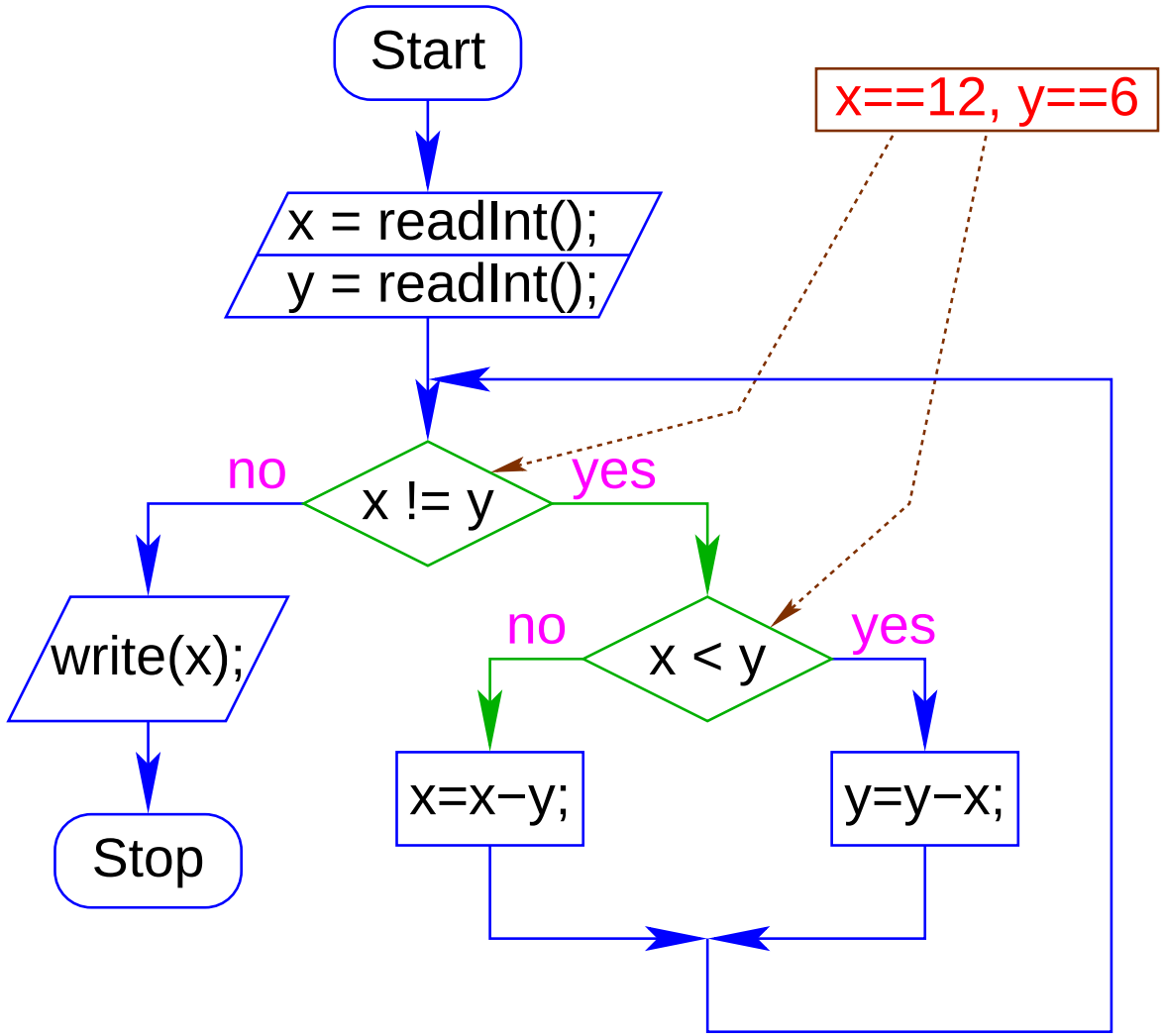

Start
x==18, y==24
x = readInt();
y = readInt();
no
x != y
yes
write(x);
no
x < y
yes
x=x−y;
y=y−x;
Stop

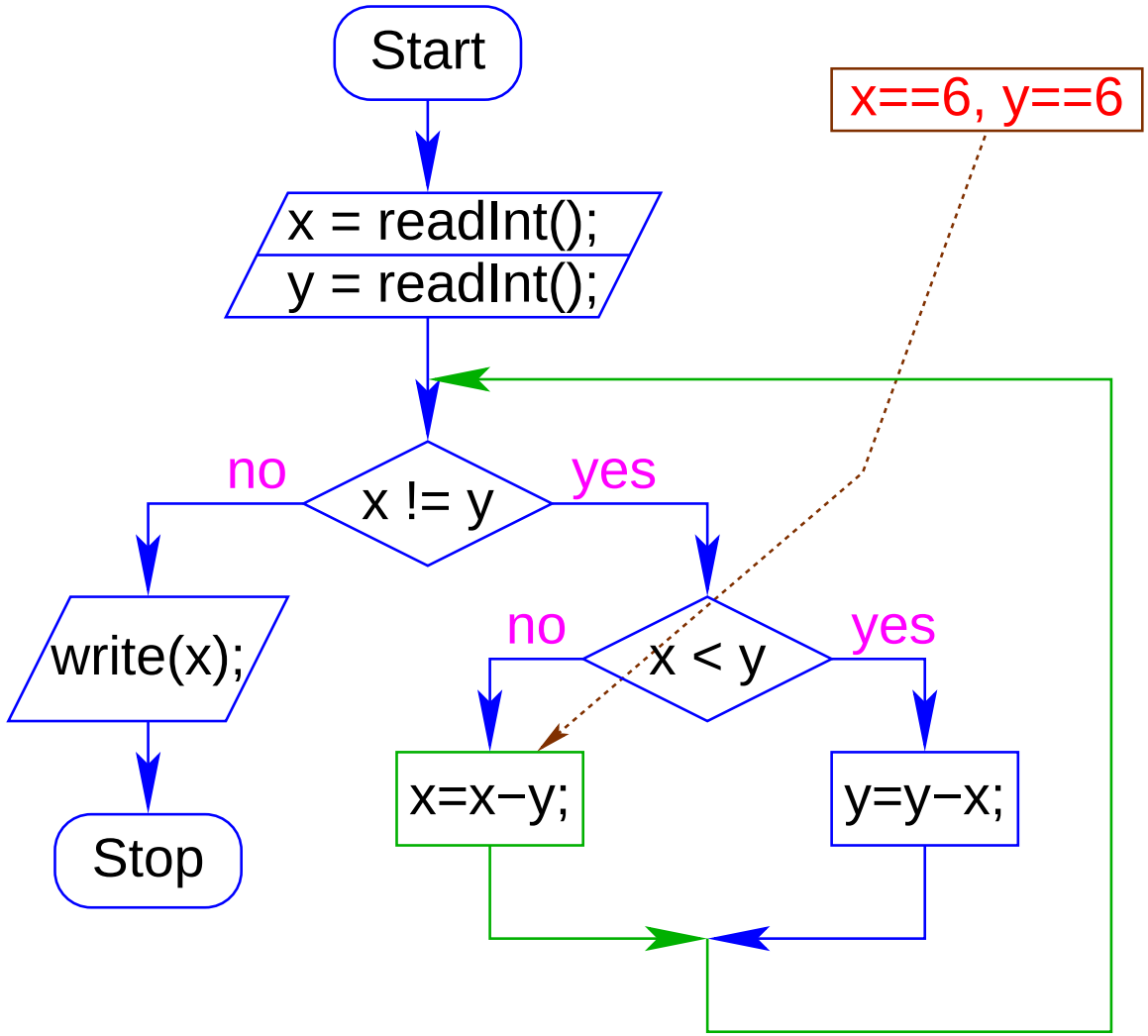

Start
x==18, y==24
x = readInt();
y = readInt();
no
x != y
yes
write(x);
no
x < y
yes
x=x−y;
y=y−x;
Stop

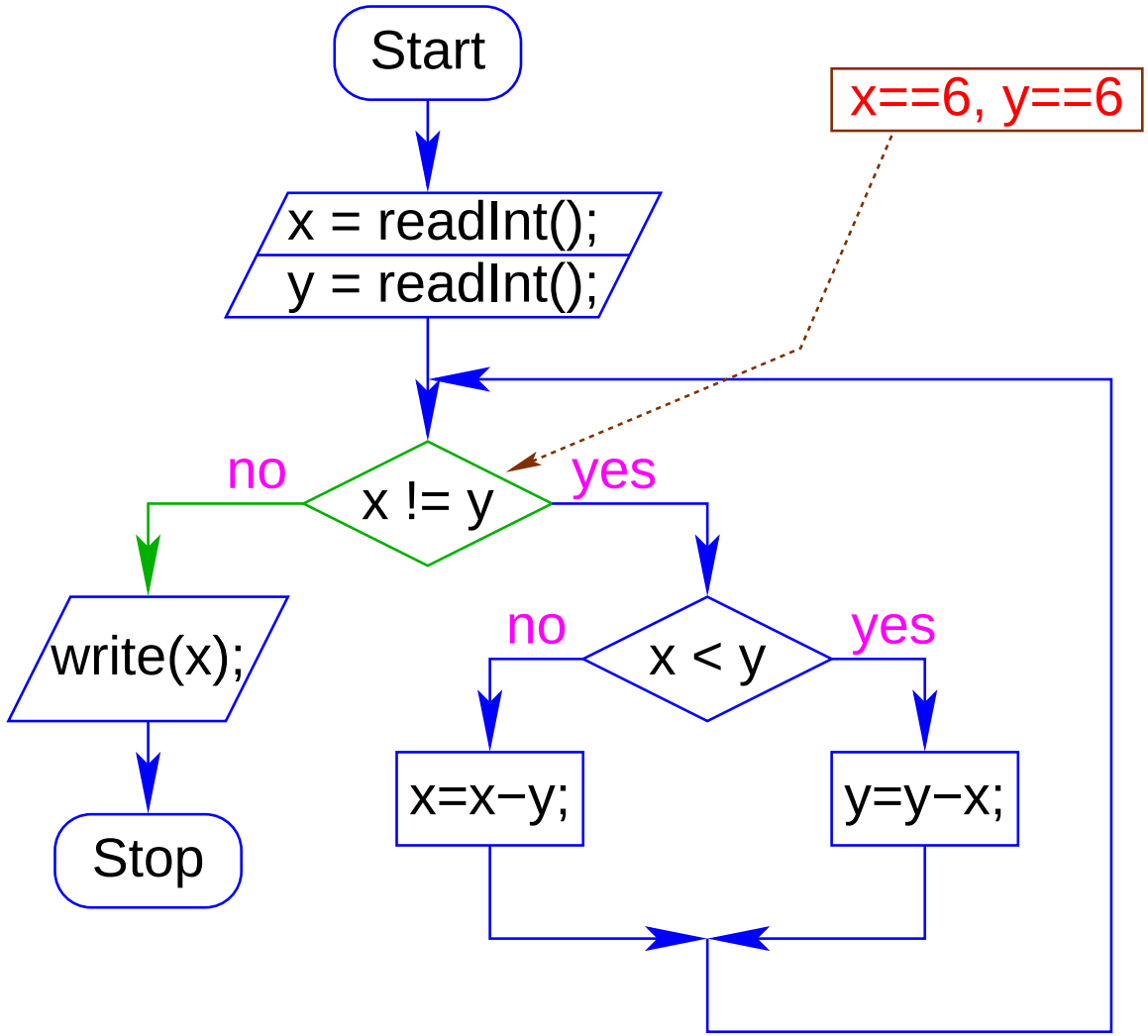

Start
x = readInt();
y = readInt();
x==18, y==6
x != y
no
yes
write(x);
Stop
x < y
no
yes
x=x−y;
y=y−x;

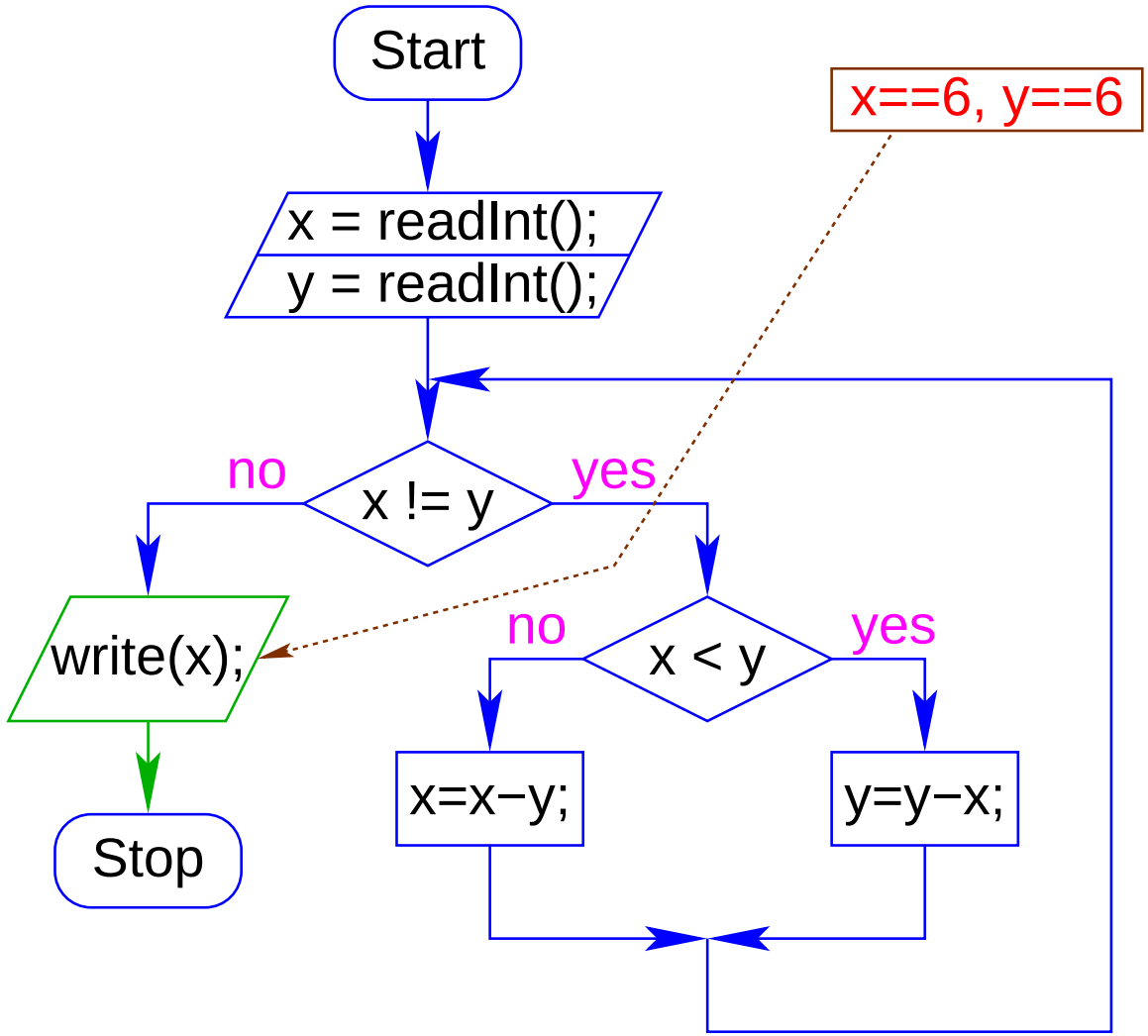

Start
x == 18, y == 6
x = readInt();
y = readInt();
no
x != y
yes
write(x);
no
x < y
yes
x=x−y;
y=y−x;
Stop

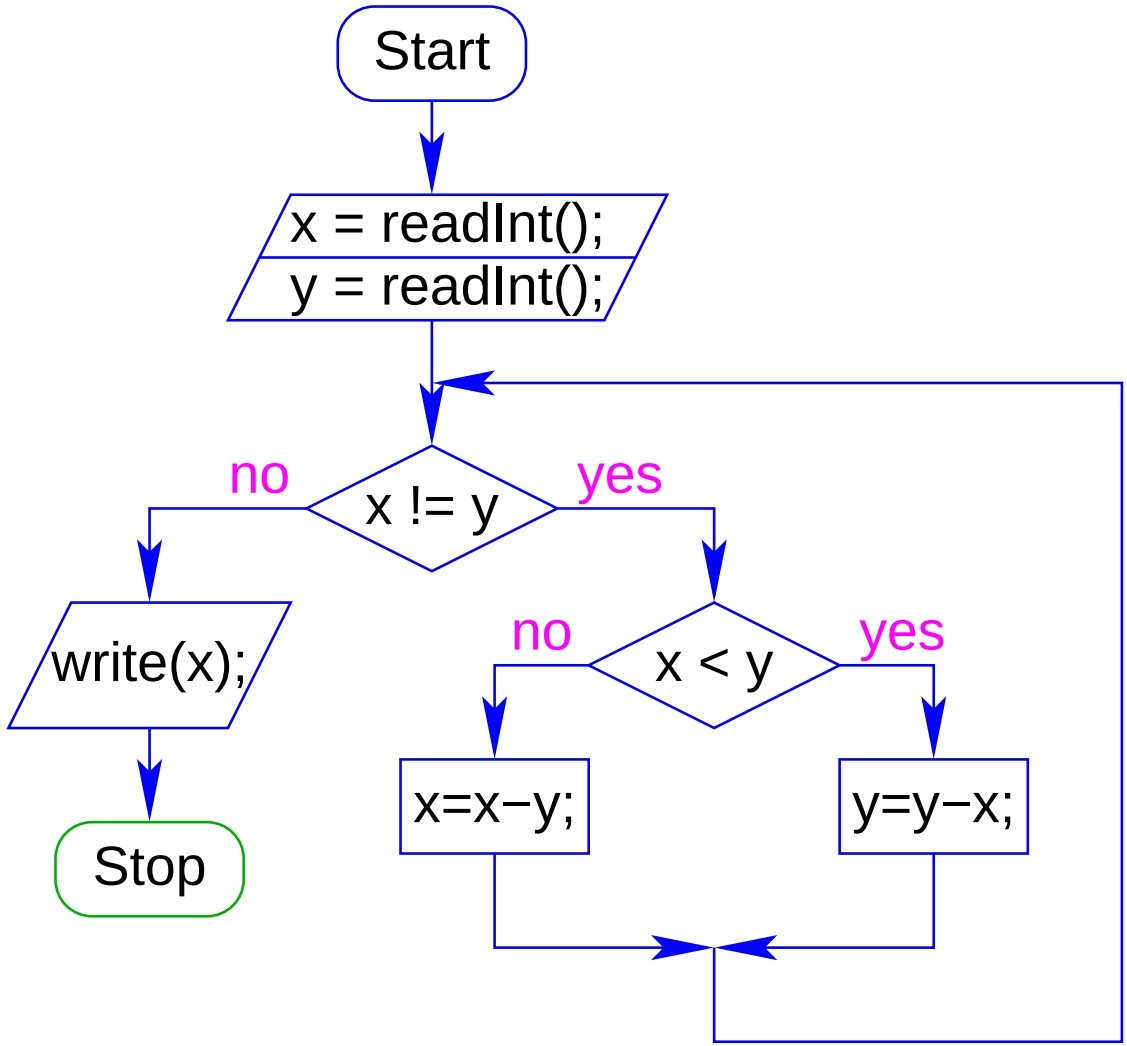

Start
x = readInt();
y = readInt();
x==12, y==6
x != y
no
yes
write(x);
Stop
x < y
no
yes
x=x−y;
y=y−x;


Start
x == 12, y == 6
x = readInt();
y = readInt();
no
x != y
yes
write(x);
no
x < y
yes
x=x−y;
y=y−x;
Stop


Start
x = readInt();
y = readInt();
x==6, y==6
x != y
no
yes
write(x);
Stop
x < y
no
yes
x=x−y;
y=y−x;


Start
x = readInt();
y = readInt();
x==6, y==6
x != y
no
yes
write(x);
Stop
x < y
no
yes
x=x−y;
y=y−x;

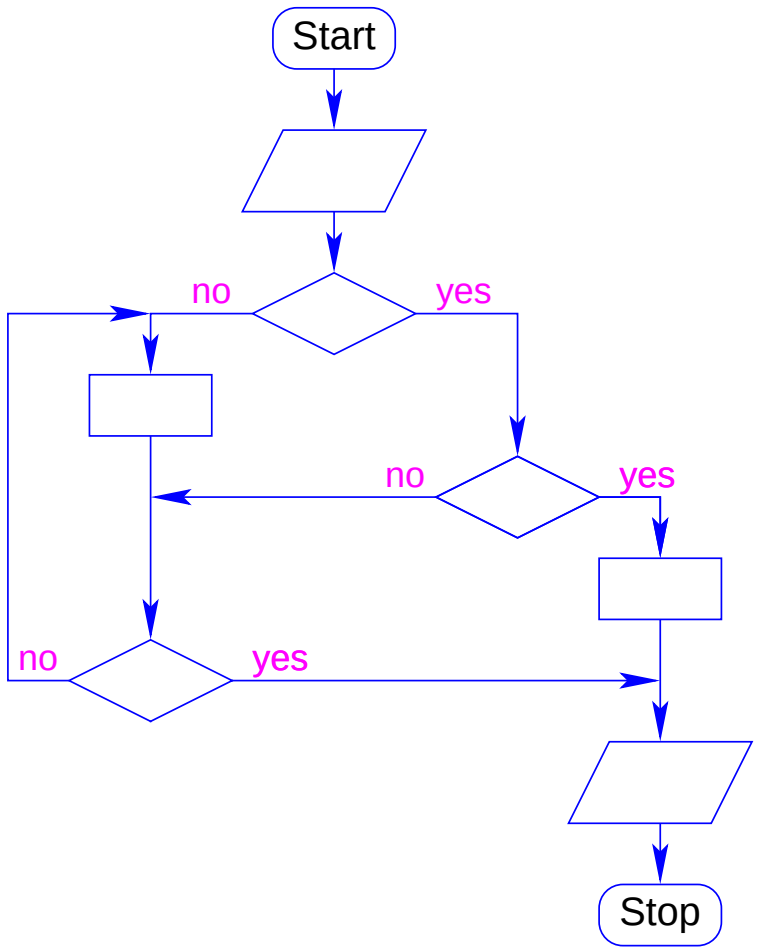

Start
x==6, y==6
x = readInt();
y = readInt();
no
x != y
yes
write(x);
no
x < y
yes
x=x−y;
y=y−x;
Stop


Start
x = readInt();
y = readInt();
x != y
no
yes
write(x);
Stop
x < y
no
yes
x=x−y;
y=y−x;

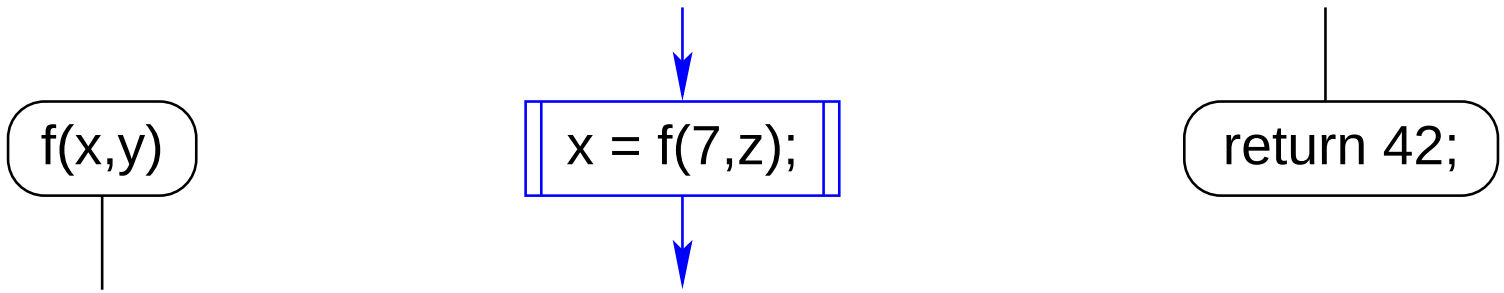
# Achtung

- Zu jedem Java-Programm lässt sich ein Kontrollfluss-Diagramm konstruieren;
- die umgekehrte Richtung gilt zwar ebenfalls, liegt aber nicht so auf der Hand.

# Beispiel



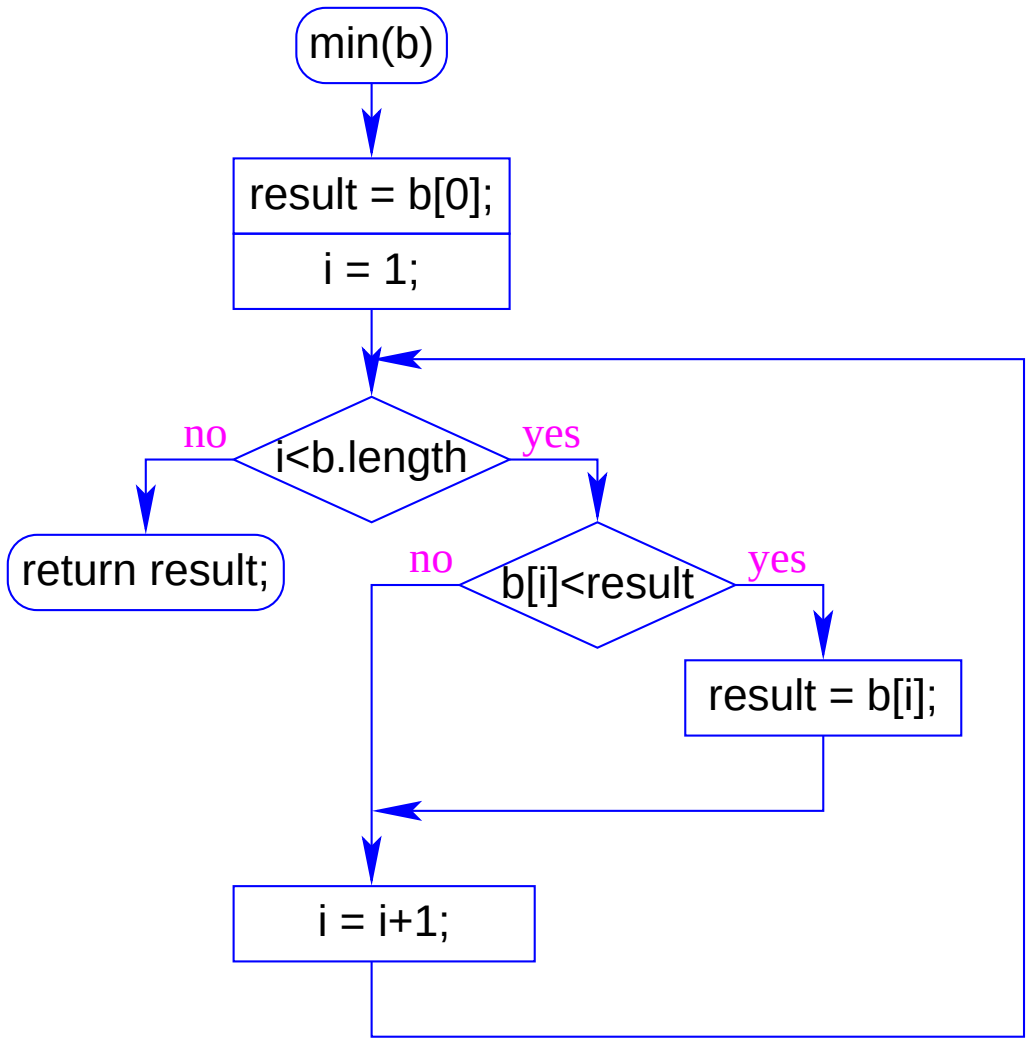
Um die Arbeitsweise von Funktionen zu veranschaulichen, erweitern/modifizieren wir die Kontrollfluss-Diagramme:



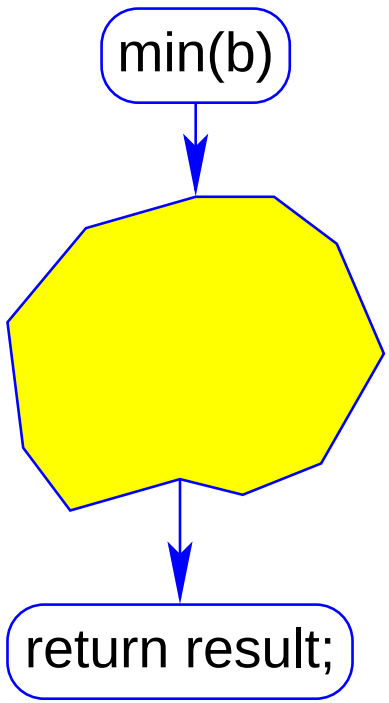

Startknoten der Funktion $f$ | Funktionsaufruf | Endknoten

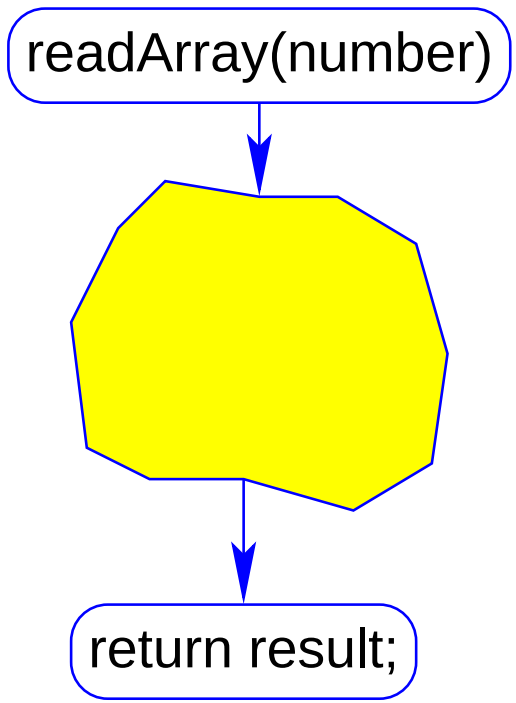
- Für jede Funktion wird ein eigenes Teildiagramm erstellt.
- Ein Aufrufknoten repäsentiert eine Teilberechnung der aufgerufenen Funktion.

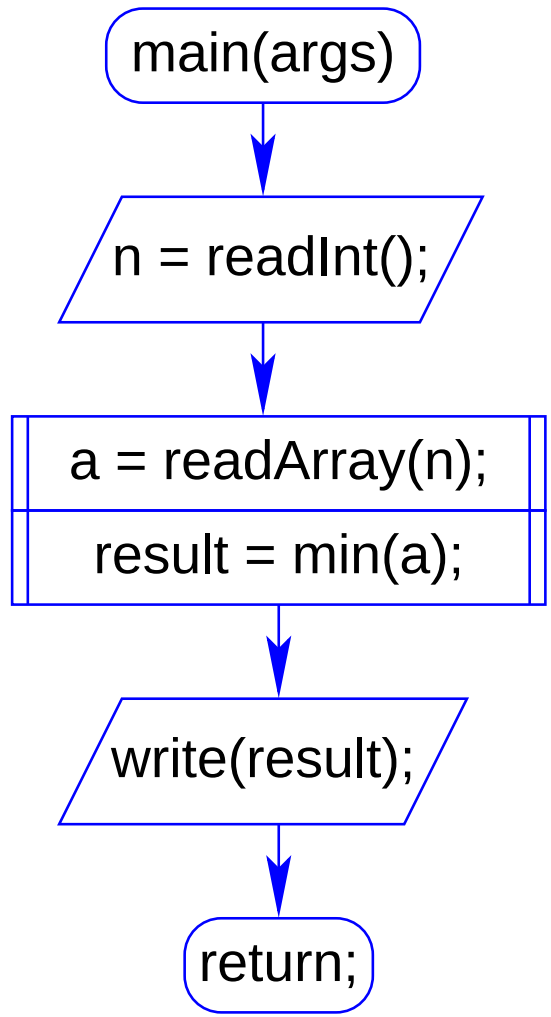
## Teildiagramm für die Funktion `min()`:

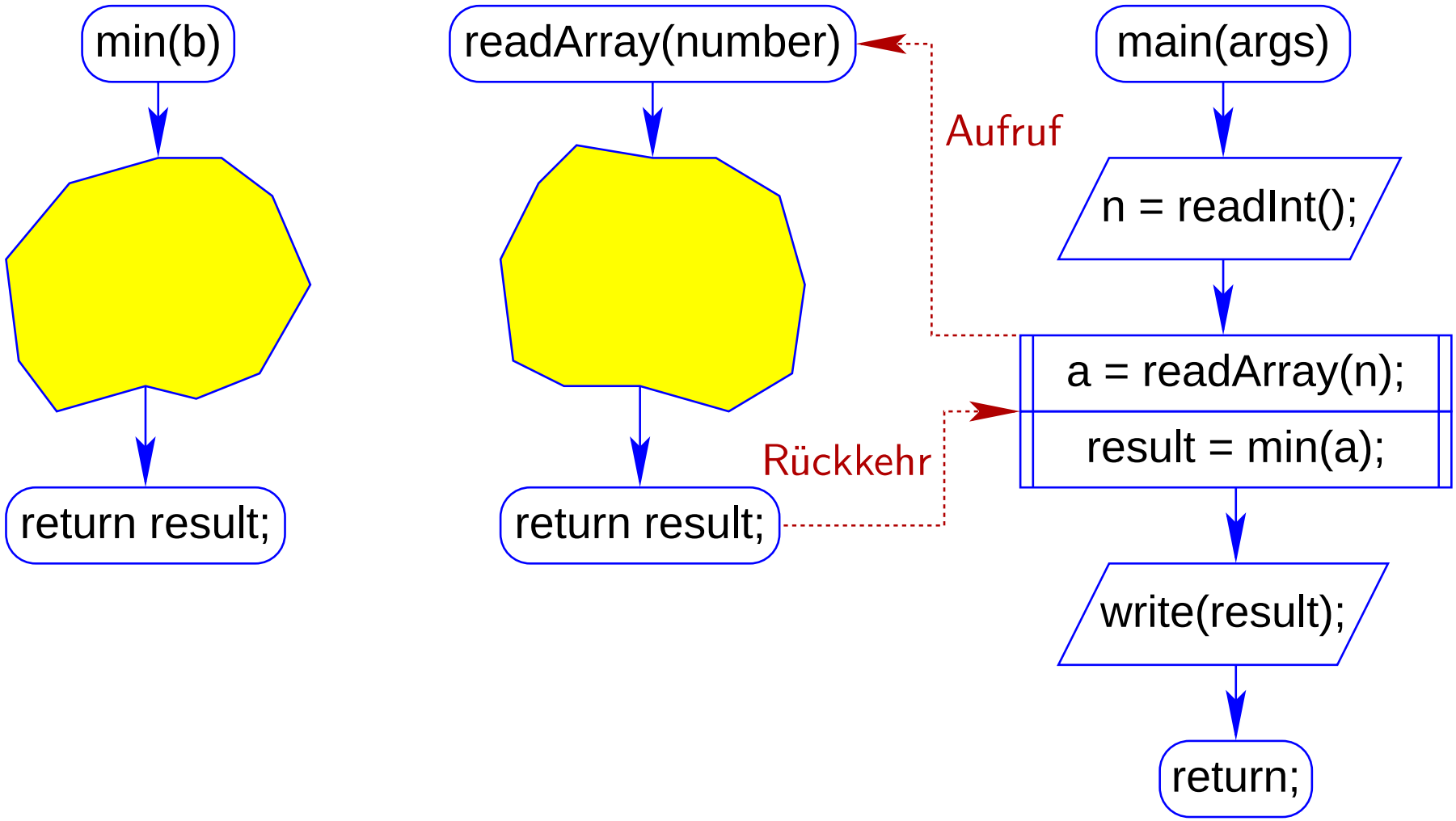


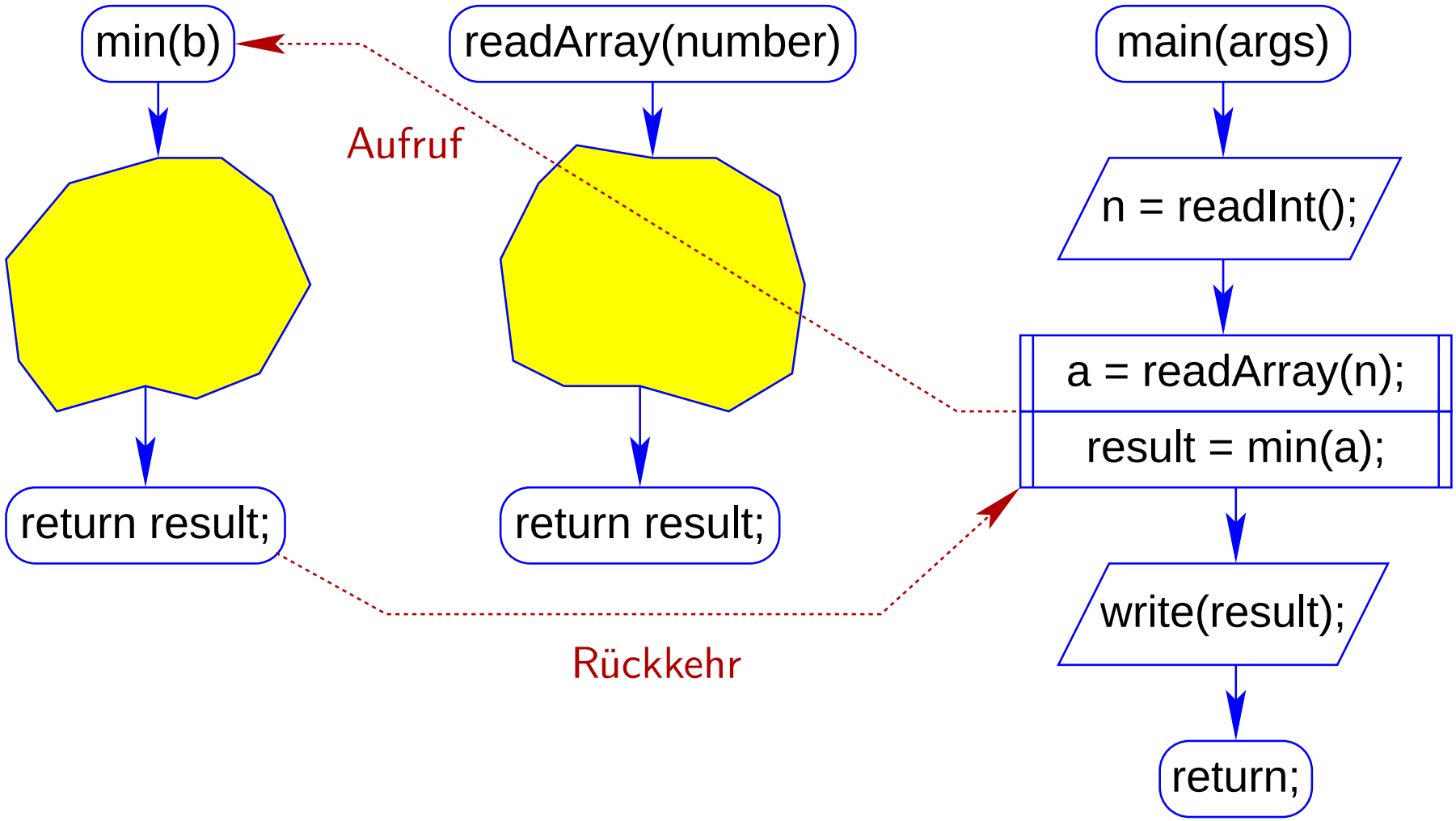
Insgesamt erhalten wir:




min(b)
return result;
readArray(number)
return result;
Aufruf
Rückkehr
main(args)
n = readInt();
a = readArray(n);
result = min(a);
write(result);
return;


min(b)
readArray(number)
main(args)
Aufruf
n = readInt();
a = readArray(n);
result = min(a);
return result;
return result;
write(result);
Rückkehr
return;